

Mobile Journal App - Technical Report

Proxie Studio - Round 1 Intern Evaluation

1. Overview

The app is a mobile journaling tool built with React Native (0.81.5) and Expo SDK 54, using React Navigation's native stack for screen flow and expo-sqlite for fully offline local storage. Users can create and edit text notes, record and play back voice notes, attach photos or files, and manage stored data from a Settings screen. Cloud sync is scaffolded in the UI but not yet connected to a backend.

2. Database Structure

Local persistence uses SQLite via expo-sqlite, opened once through a SQLiteProvider at app start. Three tables: notes, attachments (linked by note_id with cascading delete), and settings (a simple key-value store for app preferences).

Table	Column	Type	Notes
notes	id	TEXT PK	generated client-side (timestamp + random)
	title	TEXT	required
	body	TEXT	default ''
	tags	TEXT	comma-separated string
	created_at / updated_at	INTEGER	epoch ms
	synced	INTEGER	0/1, reset to 0 on every edit
attachments	id	TEXT PK	
	note_id	TEXT FK	notes.id, ON DELETE CASCADE
	type	TEXT	'image' 'file' 'audio'
	uri	TEXT	stored file NAME only, not full path
	name / mime_type	TEXT	display name, MIME type
	duration_ms	INTEGER	audio attachments only
	synced	INTEGER	0/1
settings	key / value	TEXT	key-value store, e.g. cloud_sync_enabled

Sync Status Flag

Both notes and attachments carry a synced integer flag (0/1), defaulting to 0. It resets to 0 on every edit or attachment change, so a future sync job can identify exactly which rows changed since the last upload. The Settings screen already surfaces an unsynced-item count via a getStats() query, ready to drive a sync UI once a backend exists.

Data Integrity

Saving a note and its attachment changes happens inside a single SQL transaction (saveNoteWithAttachments), so a note and its attachments are never left half-saved. Files are only deleted from disk after the database transaction commits successfully, avoiding orphaned files if a save is interrupted.

3. APIs & Libraries Used

- expo-sqlite : local relational database for notes, attachments and settings
- expo-audio : microphone recording (useAudioRecorder, HIGH_QUALITY preset) and playback (useAudioPlayer)
- expo-image-picker : camera capture and photo library selection
- expo-document-picker : picking arbitrary files from device storage
- expo-file-system (legacy API) : copying picked/recorded files from temporary cache into permanent app storage
- expo-sharing : opening/sharing attached files that aren't images or audio
- @react-navigation/native-stack : navigation between Home, Edit Note, Note Detail and Settings screens
- react-native-paper / @expo/vector-icons : UI components and iconography.

4. Key Implementation Challenges & Learnings

- Started with expo-file-system's new class-based API (File/Directory/Paths), but hit a module-resolution error on iOS in the dev preview environment. Switched to the older, more stable functional API (expo-file-system/legacy) to resolve it.
- Learned that recordings and picked files initially land in a temporary cache directory that iOS/Android can clear at any time, so they must be explicitly copied into the app's permanent Documents directory before being referenced from the database.
- Only the file NAME (not the full path) is stored in SQLite, since the app's container path can change between reinstalls or updates on iOS , the full URI is reconstructed at runtime from the known attachments directory.
- Hit an async/await bug early on: a helper function (ensureDir/saveToAppStorage) became async during the API migration, but a caller wasn't updated to await it resulting in a Promise object being stored instead of the actual file path. Fixed by propagating await through the full call chain.
- Permissions are requested just-in-time rather than at app launch: microphone access is requested right before recording starts, and camera access right before the camera opens. The photo library uses the system picker, which needs no explicit permission. A small helper (permissions.js) shows a friendly alert with a direct link to the device's Settings app when a permission is denied.
- Known issue at time of writing: the migration to the legacy file-system API is incomplete , a few functions in files.js still call the newer File class, which isn't imported under the legacy API and will throw at runtime. This needs to be finished before final submission.

5. Future Improvements

- Connect the existing "Enable cloud sync" toggle to a real backend (Supabase), using the synced flags already in place to upload only changed rows
- Weather API integration for note locations
- LLM-generated weekly summaries of journal entries
- Supabase Auth for simple login
- Passcode-based local data encryption
- Daily journaling reminders via notifications